ตราคำสาป (cursed_seal)



ขบวนมาถึงศาลาร้างและพบหลินกำลังตรวจตราหยกที่ถูกจอมมารสาป ตราที่ถูกสาปมีรหัสตรงกับ "รหัสสาป" ที่หลินแจ้งให้ฟัง อรุณต้องปลดตราเหล่านั้นออกจากสาย แล้วต่อสายของตราที่เหลือให้ติดกัน เพื่อให้ขบวนเดินทางต่อได้ คำสั่งแรกปลดเฉพาะตราแรกที่พบ คำสั่งที่สองปลดทุกตราที่มีรหัสนั้น และตราที่ถูกปลดต้องถูกทำลายทิ้ง (`delete`) ไม่ปล่อยทิ้งเป็นขยะพลังในศาลา

สิ่งที่ฝึกในด่านนี้: ลบโหนด: ตัวหัว / กลางลิสต์, ลบทุกตัวที่ตรงเงื่อนไข, `delete` โหนดที่ลบ

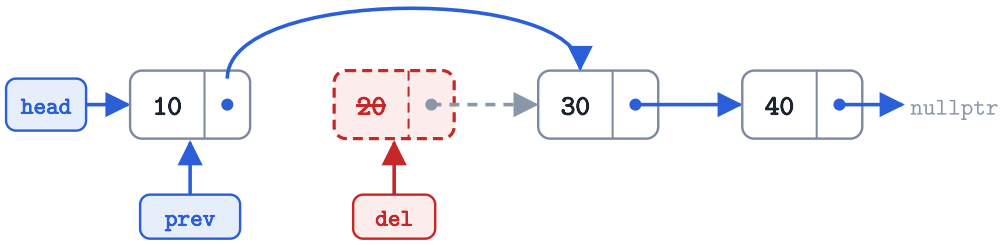
รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `cursed_seal.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

```
struct Node { int data; Node* next; };
bool removeFirst(Node*& head, int x);
int  removeAll(Node*& head, int x);
```

- `removeFirst(head, x)`: ลบและ `delete` โหนดแรกสุดที่มี `data == x` แล้วคืน `true`; ถ้าไม่พบ `x` ให้คืน `false` และไม่เปลี่ยนลิสต์ (`head` อาจเปลี่ยน หรืออาจกลายเป็น `nullptr`)
- `removeAll(head, x)`: ลบและ `delete` ทุกโหนดที่มี `data == x` แล้วคืนจำนวนโหนดที่ลบ (0 ถ้าไม่พบ)
- ห้ามสร้างโหนดใหม่ (ห้ามเรียก `new`) โหนดที่เหลือต้องเป็นโหนดเดิม เรียงลำดับเดิม และไม่ถูกแก้ `data`
- ทุกโหนดที่ถูกลบต้องถูก `delete` หนึ่งครั้งพอดี ไม่ลืม (memory leak) และไม่ซ้ำ โปรแกรมตรวจตรวจจับได้ทั้งสองกรณี
- `removeAll` ต้องเดินลิสต์รอบเดียว ใช้เวลา `O(N)` ไม่ใช่เรียก `removeFirst` ซ้ำหลายรอบ

`removeFirst(head, 20)`: ต้องหยุดที่โหนด "ก่อน" โหนดที่จะลบ (`prev`) แล้วต่อ `prev->next` ข้ามไป



`removeAll(head, 7)` ก่อนลบ: โหนดเส้นประแดงคือโหนดที่ต้องถูกลบ (มีค่าซ้ำติดกันและอยู่ที่หัวลิสต์)



หลัง `removeAll(head, 7)`: เหลือเฉพาะโหนดเดิมที่ไม่ใช่ 7 คืนค่า 3



รูปที่ 1 การลบโหนดต้องแก้ลูกศรของโหนดก่อนหน้า ส่วนโหนดแรกไม่มีโหนดก่อนหน้า ต้องแก้ `head` แทน

การส่งงาน: ดาวน์โหลดไฟล์แนบ `cursed_seal.cpp` (ไฟล์โครง) และ `cursed_seal.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `cursed_seal.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `cursed_seal.cpp` ห้ามแก้ไขไฟล์ `cursed_seal.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp cursed_seal.cpp -o cursed_seal`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียกฟังก์ชัน (ลิสต์ก่อนเรียก)	ผลที่ได้
`removeFirst(head, 8)` เมื่อลิสต์คือ $8 \to 8 \to 3$	คืน `true` ลิสต์เป็น $8 \to 3$ (ลบเฉพาะตัวแรก)
`removeFirst(head, 7)` เมื่อลิสต์คือ $2 \to 3$	คืน `false` ลิสต์ไม่เปลี่ยน
`removeFirst(head, 4)` เมื่อลิสต์คือ 4	คืน `true` และ `head` เป็น `nullptr`
`removeAll(head, 7)` เมื่อลิสต์คือ $7 \to 2 \to 7 \to 7 \to 9$	คืน 3 ลิสต์เป็น $2 \to 9$
`removeAll(head, 1)` เมื่อลิสต์คือ $2 \to 9$	คืน 0 ลิสต์ไม่เปลี่ยน

ตารางแสดงตัวอย่างการเรียกที่แยกจากกัน แต่ละแถวเริ่มจากลิสต์ตามที่ระบุ

ข้อจำกัด

- $1 \le Q \le 100$
- $0 \le N \le 200\ 000$ และ $0 \le M \le 100$ ในแต่ละชุด และ $N \times M$ ไม่เกิน 10^7
- ผลรวมของ N ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 300 000
- $-10^9 \le \texttt{data}, x \le 10^9$
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) มีเฉพาะคำสั่ง `F`, $N \le 50$ และ $M \le 50$
2. (30 คะแนน) $N \le 1\ 000$ และ $M \le 100$
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
แต่ละชุด: บรรทัดแรกมี N และ M บรรทัดถัดไปมีค่าเริ่มต้นของลิสต์ N ค่า (บรรทัดว่างถ้า $N = 0$) แล้วตามด้วยคำสั่ง M บรรทัด: `F x` เรียก `removeFirst(head, x)` · `A x` เรียก `removeAll(head, x)`
พิมพ์ 2 บรรทัดต่อหนึ่งชุด: บรรทัดแรกคือ M ตามด้วยค่าที่คืนจากแต่ละคำสั่ง (`removeFirst` พิมพ์ 1 หรือ 0 ส่วน `removeAll` พิมพ์จำนวนที่ลบ) บรรทัดที่สองคือจำนวนโหนดที่เหลือ ตามด้วยค่า `data` จากหัวถึงท้าย

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
1 4 4 8 8 3 5 F 8 F 5 F 99 F 8	4 1 1 0 1 1 3

ลบ 8 ตัวแรก $\rightarrow$ 8 3 5 · ลบ 5 (โหนดท้าย) $\rightarrow$ 8 3 · ไม่พบ 99 $\rightarrow$ ไม่เปลี่ยน · ลบ 8 (โหนดแรกของลิสต์) $\rightarrow$ เหลือ 3
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
2 5 2 7 2 7 7 9 A 7 A 4 0 1 F 1	2 3 0 2 2 9 1 0 0

ชุดแรก `removeAll` ลบ 7 ได้ 3 โหนด (รวมสองตัวที่ติดกัน) เหลือ 2 $\rightarrow$ 9 ส่วน 4 ไม่พบจึงได้ 0 · ชุดที่สองลบจากลิสต์ว่างจึงคืน `false`
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 2, 3